

Knowledge-and-Language 2025/2026

Project Report

A Bilingual Recipe Assistant Combining a Knowledge Graph, Lightweight NLP, and LLMs

Miguel Castela, uc2022212972
DEI, Universidade de Coimbra
uc2022212972@student.uc.pt

Miguel Martins, uc2022213951
DEI, Universidade de Coimbra
uc2022213951@student.uc.pt

I. ABSTRACT

We present a bilingual recipe assistant that combines a curated recipe Knowledge Graph (KG), intent classification, entity extraction and SPARQL query retrieval. This ends in an LLM generating the summary of the results in the user's language. In this report, we describe the data curation, approach, implementation, and experiments, highlighting strengths, limitations, and future directions.

II. INTRODUCTION

Motivation.

Recipe search is often unstructured and specific. Users can query for ingredients, dietary tags, or time constraints. Most systems either rely on pure keyword search or naive large language model answers. We aim to combine structured knowledge graph retrieval with natural language generation, ensuring factual retrieval and usability.

Problem Description. Given a user query in Portuguese or English, the system must:

- Detect intent;
- Extract and link entities to the KG via fuzzy matching;
- Retrieve candidate results via SPARQL;
- Perform a RAG similarity search over recipe descriptions;
- Generate summaries based on all the information, the user's query and the recent history;
- Present results through an interactive interface.

Goal and Approach Our approach is building the following pipeline:

- A curated recipe knowledge graph in RDF/Turtle;
- An intent classifier trained with BERT;
- An NLP pipeline for entity extraction and fuzzy linking;
- A FAISS embedding index for recipe descriptions;
- LLM-based summarization;
- A React/Vite-based frontend.

III. RELATED WORK

TODO

IV. DATA SELECTION

Firstly we searched for a dataset with portuguese recipes, but the ones we found were poorly structured. ([1]) is a portuguese cuisine dataset that unfortunately did not meet our requirements for structure and completeness.

Thus, we opted to use a portion of the Food.com [2] dataset that, in whole, contains over 200,000 recipes, The dataset was already curated beacuse its based on the website data that contain only recipes with the following information:

TABLE I: Fields present in each recipe entry.

Field	Description
name	Recipe name
id	Unique recipe identifier
minutes	Total preparation/cooking time
contributor_id	ID of the user who submitted the recipe
submitted	Submission date
tags	List of categorical tags
nutrition	Nutritional information vector
n_steps	Number of preparation steps
steps	Ordered list of preparation steps
description	Short user's recipe description
ingredients	List of ingredients
n_ingredients	Number of ingredients

However, as stated in the project proposal, we aim to have a bilingual dataset containing recipes in either portuguese or english languages. This is necessary to then be able to make accurate similarity retrievals and SPARQL queries based on user queries in either of these two languages.

This filtering was achieved using two methods

- Firstly, some portuguese recipes were automatically matched using a set of specific keywords defined by hand (for example: "bacalhau", "lisbon", "azores") using regular expressions. In this stage, a pre-trained langid model was also used (CITAÇÃO) to classify portuguese recipes by their full sentences, with a probability score of above 0.8. In this stage
- A lot of false negatives were still present from the first step. A pre-trained fasttext LLM (lid.176.bin)

(CITAÇÃO) was used to predict the language of each token from the data of each recipe. To be labeled as portuguese a token would have to be considered as 'PT' with a confidence ≥ 0.7 and the difference to the second detected language had to be ≤ 0.15 .

Both datasets were merged, however there was still a relatively high rate of false positives, a total of around 1435. This was partially due to a lot of spanish recipes being classified as portuguese on account of the similarity of the two languages. This problem was solved with a pass of the merged dataset through an LLM (chatGPT) (CITAÇÃO) to classify the recipes as portuguese or not portuguese.

After removing the false positives 1015 portuguese recipes remained. After this around 9000 random recipes were added to the dataset. To keep portuguese and english recipes (VER O METODO DE TIRAR RECEITAS QUE N SEJAM INGLES)

V. APPROACH

A. Knowledge Graph Construction

After the final dataset of around 10,000 recipes was obtained, the knowledge graph was generated in RDFS format. For the graph generation only the following fields were used as RDFS classes: name, id, minutes, tags, nutrition, n_steps, steps, ingredients, n_ingredients. This was the decided structured considered most relevant to build the proposed chatbot.

B. Intent Training

The first step to analyse the user's query is to detect its intent. A list of five possible intents was defined, and a dataset of around 1,100 training examples was generated using seed samples and an LLM. This dataset contained both portuguese and english examples. The following table shows some examples of user queries and their associated intents:

TABLE II: Examples of user queries and their associated intents.

User Query	Intent
Quanto tempo para cozer peito de frango	get_prep_time
Find low-carb meals	list_by_tag
What do I need for paella	retrieve_ingredients
Como fazer francesinha em casa	find_recipe
Pratos para cozinhar em 45 minutos	list_by_time

C. Entity Extraction

The next task is to extract relevant entities from the user query in order to have a base to build the SPARQL queries. This is done using a similarity search with fuzzy matching against the knowledge graph entities. The entities are extracted according to the predicted intent that has the most confidence. The entities that are considered and thus passed to the pipeline are ordered according to their similarity score, if above a certain threshold. The following table shows the mapping of intents to the entities extracted by the pipeline:

TABLE III: Mapping of intents to the entities extracted by the pipeline.

Intent	Extracted Entity / Slot
get_prep_time	recipe_name
list_by_tag	tag
retrieve_ingredients	recipe_name
find_recipe	recipe_name
list_by_ingredient	ingredient
list_by_time	minutes

D. SPARQL Queries

The valid entities extracted from the user query are then used to build SPARQL queries to retrieve the desired information from the knowledge graph. Each intent has its own SPARQL query template that is filled with the extracted entities. The following graphs shows the mapping of intents to their corresponding SPARQL query functions:

(IMAGENS DOS QUERIES)

E. Retrieval Augmented Generation

As the descriptions of the recipes in the knowledge graph are more textual and less structured, a RAG (CITAÇÃO) (retrieval augmented generation) system was implemented to be able to retrieve the most relevant descriptions based on the user query.

F. LLM Contextual Response

Finally, the retrieved information from the SPARQL queries and the RAG system is used to generate a summary response to the user query. This is done using an LLM that takes as input the user query, the retrieved recipes, and their descriptions. The LLM then generates a coherent and informative response that answers the user's query based on the retrieved information and past conversation history.

VI. IMPLEMENTATION

This section will explain in detail how each of the components described in the approach section were implemented.

A. Knowledge Graph Construction

After, we processed the dataset to generate a knowledge graph in RDFS format. The script converts a filtered recipes CSV into an RDF/Turtle knowledge graph by creating cleaned URIs and structured entities for recipes, ingredients, tags, steps, and nutrition, then serializes the result to a .ttl file. This is an example of how a recipe is represented in the knowledge graph:

```
@prefix ex: <...> .
@prefix rdf: <...> .
@prefix rdfs: <...> .
@prefix xsd: <...> .

ex:Ingredient a rdfs:Class .

ex:Nutrition a rdfs:Class .

ex:Recipe a rdfs:Class .
```

```

ex:Step a rdfs:Class .

ex:Tag a rdfs:Class .

ex:Recipe_100002 a ex:Recipe ;
  rdfs:label "chips ahoy chocolate pie" ;
  ex:hasIngredient ex:Ingredient_butter,
    ex:Ingredient_chocolate_
    chip_cookies,
  ex:hasNutrition ex:Recipe_
  100002_Nutrition ;
  ex:hasStep ex:Recipe_100002
  _StepsSeq ;
  ex:hasTag ex:Tag_60_minutes
  _or_less,
    ex:Tag_course,
  ex:id 100002 ;
  ex:minutes 40 ;
  ex:n_ingredients 5 ;
  ex:n_steps 10 ;
  ex:origin "original randomized" .

ex:Recipe_100002_Nutrition a ex:Nutrition ;
  ex:calories 3.581e+02 ;
  ex:protein 7e+00 ;
  ex:saturatedFat 6.3e+01 ;
  ex:sodium 1.5e+01 ;
  ex:sugar 4.2e+01 ;
  ex:totalFat 3.5e+01 ;
  ex:unknown 1.1e+01 .

ex:Recipe_100002_Step_1 a ex:Step ;
  rdfs:comment "crush 16 cookies in a
  ↪ sealable plastic bag until fine crumbs
  ↪ form" .

ex:Recipe_100002_Step_2 a ex:Step ;
  rdfs:comment "mix 1 1 / 2 cups of the
  ↪ crumbs with melted butter" .

ex:Recipe_100002_StepsSeq a rdf:Seq ;
  rdf:_1 ex:Recipe_100002_Step_1 ;
  rdf:_2 ex:Recipe_100002_Step_2 ;

ex:Ingredient_chocolate_chip_cookies a
  ↪ ex:Ingredient ;
  rdfs:label "chocolate chip cookies" .

ex:Tag_60_minutes_or_less a ex:Tag ;
  rdfs:label "60-minutes-or-less" .

ex:Tag_course a ex:Tag ;
  rdfs:label "course" .

```

As seen from the example above, each recipe is represented as an instance of the `Recipe` class, with properties linking to several of its components. Some of these components have their own classes, as shown below:

This structured representation allows for the reuse of entities like ingredients and tags across multiple recipes without duplication. The steps were transformed into a an RDF sequence (`rdf:Seq`) as shown in the example above and to maintain their order, each step is linked to the recipe using the `ex:hasStep`

TABLE IV: Recipe class attributes & RDFS classes.

Recipe Attributes	RDFS Classes
<code>recipe</code>	<code>Recipe</code>
<code>ingredient</code>	<code>Ingredient</code>
<code>tag</code>	<code>Tag</code>
<code>step</code>	<code>Step</code>
<code>nutrition</code>	<code>Nutrition</code>
<code>id</code>	
<code>minutes</code>	
<code>n_ingredients</code>	
<code>n_steps</code>	

property. Nutrition attributes (e.g., calories, protein, saturated fat) are encapsulated in a separate `Nutrition` instance linked to the recipe via the `ex:hasNutrition` property. Tags are linked using the `ex:hasTag` property, allowing for categorization and filtering of recipes based on various attributes. This allowed for efficient querying and easier retrieval of the relevant information for the predicted intent.

B. Intent Extraction

We defined five intents — `get_prep_time`, `list_by_tag`, `retrieve_ingredients`, `list_by_ingredient`, and `find_recipe` — and generated a dataset of around 1,100 training examples using seed samples and an LLM, ensuring clear separation between similar cases such as “recipes that take 30 minutes” (`list_by_time`) and “how long does arroz de pato take to cook?” (`get_prep_time`). Using this dataset, we trained a multilingual BERT intent classifier: it loads the labeled CSV, tokenizes the text with `bert-base-multilingual-cased`, builds label mappings, fine-tunes a sequence-classification model with appropriate hyperparameters, evaluates it on an 80/20 split, and finally saves the trained model, tokenizer, and label map. Alongside the classifier, we developed an entity extractor that uses `spaCy` for NER and `RapidFuzz` for fuzzy matching against Knowledge Graph entities, allowing downstream components to resolve ingredients, tags, recipes, and time expressions.

The `list_by_time` intent required special attention because user descriptions of preparation time are highly variable and often ambiguous. Unlike other intents that rely on explicit lexical markers, time expressions can appear in many different forms: numerical values (“30 minutos”), written numbers (“trinta minutos”), comparative forms (“menos de 20 minutos”, “under 10 minutes”), vague descriptions (“algo rápido”, “something quick”), and mixed-unit expressions such as “1h20” or “1 hora e meia”. Users may omit units, combine formats, or express constraints indirectly, which makes it difficult to rely on a single pattern or keyword. The system therefore has to distinguish between exact-time queries (asking for recipes that take exactly a specific number of minutes) and maximum-time queries (asking for recipes that take at most a certain amount of time). It also must interpret comparative structures correctly and normalize written forms

into numeric values. For these reasons, the `list_by_time` logic integrates layered numeric parsing, distinction between exact and maximum constraints, translation normalization, and fallback mechanisms so that the intent behaves consistently across differently phrased queries.

- Language detection and normalization:
 - Detects query language (pt/en) and falls back using Portuguese heuristics (accents, “ç”, common PT words).
 - Strips accents for robust matching while preserving original forms for display.
- spaCy pipeline:
 - Loads pt/en spaCy models (fallback: blank “xx”).
 - Adds an EntityRuler for TIME keywords (min/minutos/hora/hours) to improve chunking.
 - Extracts noun chunks and quoted phrases; deduplicates while preserving order.
- Cooking-time parsing (PT/EN, tolerant to typos):
 - Normalizes number words (e.g., “quarenta e cinco”, “forty-five”) into digits.
 - Uses regexes to detect minutes, hours, or combined “H+M” formats (e.g., “1h 30m”).
 - Detects range prefixes (e.g., “under”, “menos de”, “até”) to separate exact vs. max_minutes.
- Translation with protected culinary terms:
 - Uses Argos Translate offline for PT↔EN (bag-mode and dual-pass).
 - Preserves protected PT terms (e.g., “açorda”, “brás”) by reinjecting them if mistranslated.
- Candidate tokenization (`content_tokens`):
 - Removes stopwords and domain noise; sorts tokens for canonical matching.
 - Bag-mode aggregates tokens from original chunks, full text, and its translation (for ingredient/tag intents).
- Knowledge Graph indexing:
 - Loads the RDF/Turtle KG into rdflib; collects labels for recipes, ingredients, and tags.
 - Precomputes accent-stripped normalized labels for fast fuzzy search.
- Fuzzy search and linking (intent-aware):
 - RapidFuzz WRatio over normalized labels with thresholds: `ingredient=85`, `tag=90`, `recipe_name=70` (raised to 75 in fallback).
 - Portuguese label preference boost when the query appears to be PT.
 - `list_by_ingredient`: returns top ingredient matches with scores.
 - `list_by_tag`: returns tag matches and injects time-related tag synonyms (e.g., “rápido”→“15/30-minutes-or-less”).
 - `find_recipe / retrieve_ingredients`: exact/containment first; fuzzy fallback on the longest phrase.

- `list_by_time`: passes through `cooking_time` or `max_minutes` for SPARQL filtering.

- Output structure for downstream pipeline:

- Returns a dict with slots:


```
ingredient=[(label,score)], tag=[(label,score)],
recipe_name=[(label,score)],
cooking_time=int—None,
max_minutes=int—None,
plus detected_language, original_query,
translated_query.
```
- These slots feed SPARQL templates and RAG similarity; results are summarized by the LLM in the query language.

After this the retrieved information is used to generate SPARQL queries to retrieve the desired information from the knowledge graph.

INSERIR AQUI O SPARQL

This part of the pipeline is all joined in the pipeline module that does this:

C. Pipeline Orchestration (*pipeline.py*)

- Robust setup:
 - Imports the intent classifier (`predict_intent`), slot extractor (`extract_and_link`), spaCy pipeline builder, and SPARQL helper functions.
 - Loads the Knowledge Graph from `recipes_graph_cleaned.ttl` and builds a Portuguese-priority spaCy pipeline.
- Slot extraction without SPARQL:
 - `extract_slots_only(text)` predicts the top intent and extracts structured slots (ingredients, tags, recipe names, time fields) without executing any SPARQL queries.
 - Returns (`intent`, `confidence`, `slots`) for logging, debugging, and UI previews.
- Sequential query execution (`handle_query`):
 - Determines the user intent and extracts slots with `extract_and_link`, then routes execution to the appropriate SPARQL query based on the intent.
 - `list_by_ingredient`:
 - * Collects all ingredient labels from slots and runs `query_list_by_ingredient` for each.
 - * Prints formatted recipe entries (name, URI, origin, minutes, tags, ingredients, nutrition, steps).
 - * Aggregates all results into `kg_results`.
 - `list_by_tag`:
 - * Uses all extracted tag labels, executes `query_list_by_tag`, and deduplicates recipes by URI.
 - * Stores the deduplicated list in `kg_results`.
 - `find_recipe`:
 - * For each candidate recipe name, calls `query_find_recipe` and prints a full recipe card.

- * Stores results in `kg_results`.
- `retrieve_ingredients`:
 - * Calls `query_retrieve_ingredients` for each recipe name and prints the ingredient list.
 - * Appends entries to `kg_results`.
- `get_prep_time`:
 - * For each recipe name, runs `query_get_prep_time` and prints the preparation time in minutes.
 - * Aggregates results into `kg_results`.
- `list_by_time`:
 - * Reads `cooking_time` (exact) or `max_minutes` (range) from the slots.
 - * Range queries use `query_by_max_minutes`; exact queries use `query_by_exact_minutes`; sets `result_basis`.
 - * If no results or no numbers are detected, performs a fuzzy tag fallback: injects time-related tags (15/30/60-minutes-or-less), queries `query_list_by_tag`, deduplicates, and sets `result_basis` to `fuzzy_tag_fallback`.
- Formatting helpers:
 - `_fmt_recipe`: renders a compact multi-line recipe card (URI, origin, minutes, steps, ingredients, nutrition).
 - `_slot_top_label`, `_slot_all_labels`, `_slot_all_scored`: helper utilities for consistently retrieving labels and scores from slot structures.
- CLI entry-point:
 - When executed as a script, accepts `--intent-topk`, `--sparql-topk`, or `--topk`.
 - Prints query metadata: detected language, original and translated query, and detected slots before SPARQL execution.
 - Runs the sequential pipeline and prints per-intent SPARQL outputs; returns a structured dict containing `intent`, `confidence`, `text`, `slots`, and `kg_results`.

In the early versions of the pipeline we relied on a plain large-language-model translation step to normalize user queries into English before intent classification and entity extraction. While this worked reasonably well, it produced non-deterministic translations: the same Portuguese phrasing translated into English would not necessarily translate back into the same Portuguese string, and small paraphrases in the input would sometimes yield semantically equivalent but structurally different English renderings. This inconsistency became problematic because the multilingual BERT intent classifier and the spaCy-KG entity extractor depend on predictable lexical patterns. To address this, we introduced several normalization measures to make the translation stage

behave more “objective” and reproducible. First, we removed Portuguese stopwords that added noise and led the LLM to produce stylistic rather than literal translations. Second, instead of translating token by token—which caused the LLM to reinterpret fragments—we translated the entire sentence as a single unit, preserving intent and syntactic structure. Third, we fixed a systematic discrepancy in number handling: PT→EN translation tends to convert written numbers into digits (e.g., “trinta minutos” → “30 minutes”), while EN→PT does not reliably convert digits back into words. To preserve semantic symmetry, we introduced a digit-to-word mapping that rewrites numbers consistently in the canonical representation. Finally, we stored the original query, the detected language, and the normalized English form so downstream components always receive a stable and deterministic string. Together, these fixes substantially reduced variation and improved both intent classification and entity extraction accuracy. In the early versions of the pipeline we relied on a plain large-language-model translation step to normalize user queries into English before intent classification and entity extraction. While this worked reasonably well, it produced non-deterministic translations: the same Portuguese phrasing translated into English would not necessarily translate back into the same Portuguese string, and small paraphrases in the input would sometimes yield semantically equivalent but structurally different English renderings. This inconsistency became problematic because the multilingual BERT intent classifier and the spaCy-KG entity extractor depend on predictable lexical patterns. To address this, we introduced several normalization measures to make the translation stage behave more “objective” and reproducible. First, we removed Portuguese stopwords that added noise and led the LLM to produce stylistic rather than literal translations. Second, instead of translating token by token—which caused the LLM to reinterpret fragments—we translated the entire sentence as a single unit, preserving intent and syntactic structure. Third, we fixed a systematic discrepancy in number handling: PT→EN translation tends to convert written numbers into digits (e.g., “trinta minutos” → “30 minutes”), while EN→PT does not reliably convert digits back into words. To preserve semantic symmetry, we introduced a digit-to-word mapping that rewrites numbers consistently in the canonical representation. Finally, we stored the original query, the detected language, and the normalized English form so downstream components always receive a stable and deterministic string. Together, these fixes substantially reduced variation and improved both intent classification and entity extraction accuracy.

The `list_by_time` intent required special attention because user descriptions of preparation time are highly variable and often ambiguous. Unlike other intents that rely on explicit lexical markers, time expressions can appear in many different forms: numerical values (“30 minutos”), written numbers (“trinta minutos”), comparative forms (“menos de 20 minutos”, “under 10 minutes”), vague descriptions (“algo rápido”, “something quick”), and mixed-unit expressions such as “1h20” or “1 hora e meia”. Users may omit units, combine

formats, or express constraints indirectly, which makes it difficult to rely on a single pattern or keyword. The system therefore has to distinguish between exact-time queries (asking for recipes that take exactly a specific number of minutes) and maximum-time queries (asking for recipes that take at most a certain amount of time). It also must interpret comparative structures correctly and normalize written forms into numeric values. For these reasons, the `list_by_time` logic integrates layered numeric parsing, distinction between exact and maximum constraints, translation normalization, and fallback mechanisms so that the intent behaves consistently across differently phrased queries.

Paragraph: Initial translation problems and motivation for fixes

In the early versions of the pipeline, we relied on a plain large-language-model translation step to normalize user queries into English before running intent classification and entity extraction. Although this approach worked on a superficial level, it produced non-deterministic and sometimes unstable translations: identical Portuguese phrasing could generate slightly different English outputs, and paraphrased inputs could result in inconsistent structural transformations. This unpredictability created problems because both the multilingual BERT intent classifier and the spaCy-based entity extractor rely on lexical regularity and stable patterns. Moreover, number expressions behaved differently across translation directions: Portuguese written numbers were often converted to digits in PT→EN, but EN→PT translation would not always preserve the same normalization. To address these issues, the translation stage was redesigned to be more “objective,” removing sources of randomness and enforcing uniform pre-processing rules.

Paragraph: Explanation of the fixes implemented

To mitigate the inconsistencies introduced by LLM-based translation, we introduced several normalization steps that force the system to behave deterministically. First, we added stopword removal to reduce noise and avoid subtle LLM paraphrasing triggered by function words. Second, instead of translating token-by-token or in fragments—which increases variability—we translate the entire user sentence as a single unit, which preserves structure and meaning more reliably. Third, because written Portuguese numbers were being converted into digits on the PT→EN pass but not symmetrically handled on the EN→PT pass, we implemented a dictionary mapping written numbers to their numeric forms so that normalization becomes rule-based rather than dependent on translation behavior. Finally, we added additional deterministic parsing logic, visible in functions such as `parse_time_constraints()` and `normalize_pt_numbers()`, to ensure that mixed formats like “1h30”, “1 hora”, or “vinte e cinco minutos” are always converted into consistent numerical outputs. Together, these modifications eliminate translation drift and ensure that intent detection and slot extraction operate on stable, predictable inputs.

Python references using . . . for LaTeX

Here are the relevant code identifiers wrapped for LaTeX:

```
parse_time_constraints()
normalize_pt_numbers()
link_candidates_to_kg()
extract_and_link()
cooking_time_raw
max_minutes_raw
RANGE_PREFIX_REGEX
```

2. Pre-loading heavy components (spaCy pipeline + KG index + PKL TTL loader)

A crucial architectural decision in the pipeline is the pre-loading of all heavy components—namely the multilingual spaCy pipeline and the knowledge-graph index—before any user query is processed. The spaCy model initialization involves loading statistical components, taggers, dependency parsers, custom extension attributes, and multilingual vocabulary vectors; performing this at runtime for every query would be prohibitively expensive. Similarly, the knowledge graph, sourced from a Turtle file exceeding one million lines, cannot be reparsed from RDF on demand. To address this, the pipeline implements a PKL-based caching system: on the first run, the TTL file is parsed into an in-memory graph structure, which is then serialized to disk as a .pkl snapshot. Subsequent executions detect the presence of the pickle file and load the graph directly in a fraction of the time. This strategy not only improves performance dramatically but also stabilizes memory usage, since the expensive graph structure is built exactly once and then reused. By storing these pre-loaded components in global module scope, the pipeline mirrors the deployment model of mature production NLP systems, where models and indexes behave as long-lived, immutable dependencies, ensuring deterministic processing and consistent latency during interactive use.

3. Intent-dependent slot extraction

The pipeline introduces a dedicated mechanism that controls when entity extraction is executed, preventing unnecessary or harmful parsing for intents that do not require semantic grounding. This is achieved through the set `INTENTS_NEEDING_EXTRACTION`, which acts as a whitelist of intents for which slot extraction using spaCy and the knowledge graph is meaningful. For intents such as greetings, confirmations, or generic help requests, attempting entity extraction would lead to spurious results, noise in the entity linker, and contradictions between the detected intent and the returned slot set. By gating extraction behind an explicit membership check, the pipeline ensures that computational resources are used only when needed while also improving the semantic cleanliness of downstream processing. This design aligns with standard NLU frameworks such as Rasa, Snips, and Dialogflow, where entity extraction is conditioned on the intent rather than blindly applied to all utterances. The result is a more predictable and efficient NLU layer that avoids over-generation and keeps intent–slot coherence intact.

4. Clear separation between “slot extraction only” and full pipeline execution

Another important structural decision in the codebase is the strict separation between two main operations: NLU-only processing and full end-to-end query resolution. The function `extract_slots_only(text)` performs intent classification and optional entity extraction but halts before invoking any SPARQL logic, making it ideal for debugging, unit testing, and batch evaluation of the NLU layer without the noise of the knowledge-graph back-end. In contrast, `handle_query(text)` performs the complete pipeline: classification, slot extraction, SPARQL execution, aggregation of results, and formatted output. This separation of responsibilities reflects a widely adopted architecture in dialogue systems, where the NLU module is distinct from the dialogue manager or API layer. It also facilitates integration into GUIs or external services, which may only need intent and slot information and would be disrupted by the verbose SPARQL printing the full pipeline produces. By keeping both modes available, the system gains flexibility, debuggability, and modularity, allowing each component to be validated in isolation.

6. De-duplication logic for overlapping recipe results

The knowledge graph’s structure and tagging system make it possible for a single recipe to satisfy multiple filtering criteria at once—for example, a dish might be labeled both as “quick” and “under 30 minutes,” or appear in multiple result sets inferred through different SPARQL paths. Without explicit handling, such overlaps would cause duplicated recipe entries in the output, misleading users and complicating evaluation of recall and precision. To solve this issue, the pipeline applies a deterministic de-duplication pass after SPARQL results are collected. It iterates over the returned recipes, tracks seen URIs in a set, and retains only the first occurrence of each unique recipe. This lightweight but effective mechanism guarantees that results remain clean, non-redundant, and semantically meaningful. The decision to deduplicate at the pipeline level rather than the SPARQL level preserves query flexibility: the SPARQL routines remain simple and independent, while the consolidation logic ensures their results form a coherent whole.

VII. RAG IMPLEMENTATION

This module builds and serves a vector-based semantic search index over recipe descriptions. It loads a curated bilingual CSV file containing recipe IDs, names, and descriptions. Before indexing, the code applies a quality-filtering step: short, boilerplate, or low-information descriptions are detected using heuristics such as length, alphabetic-character ratio, presence of boilerplate patterns (e.g., prep time, yield), and token count. Any description that fails these checks is skipped. Valid descriptions are then turned into “chunks”—in this version, each description is a single chunk—which are stored in a list of records containing fields like `{"id": ..., "name": ..., "chunk_id": ..., "text": ...}`. The build step encodes all chunks using a SentenceTransformer model. The model name is configurable via an environment variable, defaulting to `intfloat/multilingual-e5-base`; if the selected

model is an E5 model, the code automatically prepends the prefix `"passage: "` to each description, following the E5 formatting convention. The resulting embeddings are L2-normalized and saved to `embeddings.npy`. A FAISS inner-product index is then created and written to `faiss.index`. Alongside it, a metadata file `desc_meta.json` is generated, containing fields such as `{"records": [...], "meta": {"model": "...", "use_e5": true}}`, stored via `\verb`—so it can be used later for querying. When querying, the index and metadata are lazily loaded and cached. A query string is encoded using the same model, applying the prefix `"query: "` if E5 is active. The FAISS index retrieves the most similar chunks; extra candidates are fetched to allow deduplication by their `chunk_id`. Results below a similarity threshold are filtered out. The returned list includes the recipe ID, recipe name, chunk ID, the original description text, and the similarity score. Several helper functions enhance retrieval. The function `best_description_for_names` returns stored descriptions for a given list of recipe names. The function `_augment_with_descriptions` enriches an intent-extraction result object by attaching two fields: `descriptions_by_name`, which maps detected recipe names to their best descriptions, and `similar_description_chunks`, which provides up to the top 3 most semantically similar description chunks for the user’s entire query. Finally, running the module directly (`python module.py`) triggers the index-building process.

VIII. SYSTEM ARCHITECTURE

Initial tests with the local Ollama model proved unsatisfactory. Despite the advantage of running locally without relying on an external service, the models demonstrated semantic inconsistencies, context drift, and high variability in intent and label extraction. The low latency was not enough to compensate for the poor response quality. Consequently, the implementation was migrated to the Groq API, which provided significant improvements in response accuracy, coherence across turns, and overall stability. The Groq client is initialized lazily, ensuring that the module is loaded only when required. This section describes the evolution of the LLM query module implementation, including the limitations encountered with Ollama, the subsequent migration to the Groq API, the mechanism for tracking the last messages, the *prompt engineering* strategy, the pipeline import process via `pickle`, the command-line interface (CLI), as well as the *prompt trimming* system to ensure the 7000-token limit is not exceeded.

9. Testing with Ollama and Migration to Groq

Initially, tests were conducted with the local Ollama model. Despite the advantage of not relying on an external service, these tests were unsatisfactory. The models exhibited semantic inconsistencies, context drift, and high variability in intent and label extraction. Latency was low, but the response quality was insufficient for the level of precision required by the project. Therefore, a migration to the Groq service was carried out.

This transition significantly improved response quality, coherence across turns, and model stability. The integration now uses a Groq client initialized lazily (*lazy init*), ensuring the module is only loaded when actually needed. The initialization logic is encapsulated in the client:

```
_groq_client = None

def _ensure_client():
    from groq import Groq
    return Groq(api_key=GROQ_API_KEY)
```

Conversation History of Recent Messages

To avoid exceeding the per-minute request limit and reduce computational costs, a compact history of the last three messages was implemented. Only three fields are stored:

- `query` – the original user question,
- `structured` – the structured block generated by the semantic pipeline,
- `response` – the final response returned by the LLM.

This history is used to generate a reduced context block appended at the end of the *prompt*. The format is defined as:

```
_conversation_history: list[dict[str, str]]
CONTEXT_WINDOW = 3
```

And rendered with:

```
def _build_context_annotation(prev_turns):
    ...
```

Placing the history at the end — rather than at the beginning — ensures that the trimming system always cuts from the tail, preserving the most relevant part of the main prompt.

Prompt Engineering

The section responsible for prompt construction integrates:

- structured summary of the pipeline result;
- similarity *snippets* from the RAG index;
- assistant behavioral decision rules;
- compact block of recent message history.

This block is generated by the function:

```
_summary, full_prompt_sent = _summarize_with_llm(user_query, response)
```

The *prompt engineering* includes:

- responding always in the original query language,
- modularization of responses by entities (ingredients, tags, names),
- detection of meta-queries (“what did I ask before?”),
- safe fallback when there is noise or off-topic content,
- conservative use of semantic snippets (RAG).

Pipeline Import (Pickle, TTL, Lazy Loading)

The semantic pipeline is loaded only when necessary, avoiding startup costs:

```
mod = importlib.import_module("NLP.pipeline")

def _load_pipeline():
```

```
    _handle_query = getattr(mod, "handle_query")
```

The components are pre-serialized in a TTL file (ontology) and pickle files to accelerate initialization.

CLI Information

The module exposes a fully interactive command-line mode:

Interactive Recipe Chatbot (Pipeline + Groq)
Type a recipe-related query in PT or EN.
Commands: /exit to quit

Features include:

- direct execution: `python module.py "recipes with tofu"`
- interactive mode: continuous input;
- `clear` command to reset internal history;
- non-interactive `stdin` support (useful for scripts).

Automatic Prompt Trimming (7000-Token Limit)

To avoid context overflow errors, a trimming system based on approximate token count (via `tiktoken`) was implemented. The default behavior is:

- calculate the tokens of the full prompt;
- if it exceeds 7000, always trim from the end (tail);
- repeat successive 20% cuts until within the limit.

The logic:

```
def _trim_prompt(prompt, max_tokens, model):
    cut_len = int(len(current) * 0.80)
    current = current[:cut_len]
```

This strategy preserves the most important part of the prompt: the main instruction block and the structured pipeline summary, removing only the older history.

Overall Code Description

The module coordinates:

- conditional loading of environment variables using `dotenv`;
- lazy loading of the semantic pipeline;
- integration with the RAG similarity engine;
- prompt construction with history and response policy;
- sending requests to Groq via `client.chat.completions`;
- logging conversation history in `context.txt`;
- maintaining an interactive session in the CLI.

The main workflow occurs in:

```
def _process_query(user_query):
    handle_query_fn, _ = _load_pipeline()
    result = handle_query_fn(...)
    summary, used_prompt = _summarize_with_llm(...)
    return summary
```

A. Frontend details

The frontend of the recipe chatbot was designed with several key user-focused features. Users can create a new chat, which clears the message history stored in the Groq prompt, ensuring that subsequent queries start with a fresh context. While a new chat is loading, the interface allows users to view the previous message history for reference. The displayed text on the interface can also be dynamically translated, independent of the chatbot response language, which is automatically matched to the query’s language. Additionally, all LLM responses are converted to Markdown for rendering, enabling proper formatting of tables and structured content. These frontend features are integrated with the backend described earlier, which is exposed via a FastAPI server. The API supports endpoints for health checks, submitting chat queries, and clearing the conversation context. The FastAPI backend ensures that the semantic pipeline is loaded once, lazily, and that all requests—including clearing the conversation history—are handled asynchronously for a responsive user experience. The frontend itself is implemented in React, with a dedicated `fonts` and `components` folder to manage styling and reusable UI elements.

IX. EXPERIMENTATION

A. Setup

We evaluated PT/EN queries by ingredients, tags, and time constraints.

B. Results

Positive cases showed accurate intent routing and bilingual output. Initial failures in time parsing and API configuration were mitigated through relaxed matching and environment validation.

Resultados

- Mostrar o quão boas as SPARQL queries foram a buscar os resultados desejados do dataset - resultado
- Refletir a introdução do RAG para a descrição no sistema, mesmo com as melhorias. - resultado
- Mostrar a interpretação das mensagens do historico + prompt + resultados do groq (receitas/ingredients/tags/steps/time pedidos de acordo com a query) - uma tablea com exeplos na parte do prompt engineering

C. Metrics

- Intent accuracy (placeholder);
- Entity linking precision@k (placeholder);
- Time extraction success rate (placeholder);
- Human quality judgments for bilingual summaries (placeholder).

X. CONCLUSIONS AND FUTURE WORK

We demonstrated that combining symbolic reasoning with neural generation produces accurate and fluent responses. The bilingual KG-powered assistant effectively supports multiple query types.

Future work includes:

- expanding KG coverage;
- adding automatic evaluation metrics;
- improving translation robustness;
- integrating structured outputs.

REFERENCES

- [1] C. Antelo, “Portuguese meals dataset,” <https://www.kaggle.com/datasets/catarinaantelo/portuguese-meals>, 2021, kaggle dataset; Accessed: 2025-12-04.
- [2] S. Li, “Food.com recipes and interactions dataset,” <https://www.kaggle.com/datasets/shuyangli94/food-com-recipes-and-user-interactions>, 2020, accessed: 2024-06-10.